

Metrics for Business Process Models

Up until now, there has been little research on why people introduce errors in real-world business process models. In a more general context, Simon [404] points to the limitations of cognitive capabilities and concludes that humans act rationally only to a certain extent. Concerning modeling errors, this argument would imply that human modelers lose track of the interrelations of large and complex models due to their limited cognitive capabilities and introduce errors that they would not insert in a small model. A recent study by *Mendling et al.* [275] explores in how far certain complexity metrics of business process models have the potential to serve as error determinants. The authors conclude that complexity indeed appears to have an impact on error probability. Before we can test such a hypothesis in a more general setting, we have to establish an understanding of how we can define determinants that drive error probability and how we can measure them.

In this instance, measurement refers to a process that assigns numbers or symbols to attributes of entities in the real world [124] in order to represent the amount or degree of those attributes possessed by the entities [432, p.19]. This way, measurement opens abstract concepts to an empirical evaluation and is, therefore, a cornerstone of natural, social and engineering sciences. In this definition, an attribute refers to a property or a feature of an entity while an entity may be an object or an event in the real world. Measurement at least serves the following three purposes: understanding, control and improvement. The classical statement attributed to *Galilei*, “What is not measurable make measurable”, stresses the ability of a measurement to deliver *understanding*. The principle idea behind this phrase is that measurement makes concepts more visible. In effect, entities and their relationships can be tracked more precisely bringing forth a better understanding. In an emerging discipline like complexity of business process models, it might not be clear what to measure in the first place. Proposing and discussing measures opens a debate that ultimately leads to a greater understanding [124, p.7]. Measurement then enables *control* in order to meet goals. According to *DeMarco* “you cannot control what you cannot measure” [103]. Based on an understanding of relationships between different attributes, one can make predictions such as whether goals will be met and what actions need to be taken. For business process modeling projects, it is important to establish suit-

able measurements since, as *Gilb* points out, projects without clear goals will not achieve their goals clearly [141]. The lack of measurements that can be automatically calculated from a given process model is a central problem of several quality frameworks. Examples include the Guidelines of Modeling (GoM) [50, 388, 51], SEQUAL [250, 307, 228] or the work of *Güceglioglu and Demirörs* [157, 156]. While various empirical research has been conducted on quality aspects of data models (see [304, 135, 305, 136, 137]), such work is mostly missing for business process models [306]. Defining quality concepts in a measurable way would be a major step towards understanding bad process design in general. Measurement is also crucial for the *improvement* of both business process models as products and business process modeling processes. In business science, it is an agreed upon insight, from *Taylor's* scientific management [421] to *Kaplan and Norton's* balanced scorecard [205, 206], that measurement directs human behavior towards a goal. In organizational theory, this phenomenon was first recognized in the Hawthorne plant of Western Electric and is referred to as the Hawthorne Effect: what you measure will be improved (see [234, p.21]). Business process modeling has not yet established a general suite of measurements that is commonly accepted. The potential for improvements in current modeling practices is, therefore, difficult to assess. This chapter aims to contribute to a more quantitatively oriented approach to business process modeling by proposing a set of potential error determinants for EPC business process models. This is also a step towards establishing business process modeling as an engineering discipline since “to predict and control effectively you must be able to measure. To understand and evaluate quality, you must be able to measure.” [234, p.4].

The remainder of this chapter is structured as follows: Section 4.1 presents the theoretical background of measurement with a focus on scale types and issues related to validity and reliability. Section 4.2 discusses which concepts are measured in the neighboring discipline of network analysis. We focus on degree, density, centrality and connectivity metrics since they seem to be promising for business process models. Section 4.3 gives an overview of complexity metrics in software engineering. We highlight the most prominent metrics and discuss their relationship to more abstract quality concepts for software products. In Section 4.4, we present related work on metrics for business process models. In Section 4.5, we identify the complexity of a process model's structure and its state space as the key determinants for error probability. Related to these two aspects we define a set of metrics and discuss their impact on error probability. Section 4.7 gives a summary before the metrics are tested in the subsequent chapter.

4.1 Measurement Theory

The representational theory of measurement (or measurement theory) explains how empirical phenomena can be captured in terms of a measurement (see [474, 124, 409]). A measurement can be formally defined as a mapping (also called *scale*) from the domain of the empirical world to the range of mathematical concepts (see [124]). In software engineering, the terms metric and measurement are often used

interchangeably. There seems to be a certain reluctance to make a clear distinction between both terms, partially to avoid confusion with the term *metric* in a mathematical sense (see [124, p.103]). Several software engineering books only give vague characteristics of a metric (see [140, p.368], [413, p.567] or [234, p.9]). Throughout the remainder of this book we will use the term *metric* in order to refer to the *type* of a measurement that quantifies a certain attribute of an entity in the real world by following a predefined way of quantification. An implication of this definition is that entities of the real world can be compared by comparing the measurement for a certain metric. The term *statistic* is related to such an understanding of a metric since it refers to a sample as a specific entity of the real world [20, p.212]. A sample can be defined as a subset of measurements selected from a population while a population consists of the set of all measurements in which an investigator is interested [20, p.17].

Three problems related to measurement have to be considered (see [439]). First, the *representation problem* relates to the condition that the mapping should preserve relations that exist in the real world. Second, the *uniqueness problem* refers to invariance of a measurement under certain transformations. Third, the *meaningfulness problem* is concerned with drawing conclusions about the truth of statements based on comparison of assigned scale values. While the third aspect is subject to ongoing research, the first and the second problem are addressed by the scale hierarchy proposed by *Stevens* [416]. It distinguishes nominal, ordinal, interval and ratio scales. *Stevens* assumes that a scale maps an empirical relation to the real numbers and discusses how the scale values can be interpreted.

- *Nominal*: The values of this scale can only be interpreted as unique identifiers. Consider, for example, two business process models that have the unique IDs 1 and 7. The only conclusion that can be drawn from this data is that the models are not the same. Any transformation of the scale values can, therefore, be performed if the uniqueness of the identifiers is not affected.
- *Ordinal*: The ordinal scale preserves an order relation that exists in the domain of the empirical relation. For a questionnaire asking in how far several business process models are complex, the responses could be mapped to the scale values 1 for trivial, 2 for rather simple, 3 for rather complex and 4 for incomprehensible. This representation of the responses is as good as any other that does not change the order of the values. Any monotone transformation like taking logs or square roots can be applied.
- *Interval*: The interval scale is invariant to linear transformations that preserve relative distance. Consider two business process models and the point in time when they were designed. We could encode the values as days after the start of the modeling project (01 September 2000 for example) or as years AD in the range of real numbers. We could then use a linear transformation to calculate from one scale to the other.
- *Ratio*: In contrast to the interval scale, the ratio scale has a zero element that represents absence of a certain property. Consider the number of connectors a certain business process model has. The values can accordingly be multiplied by a constant as this preserves the relative ratio of the values.

There is a hierarchy between the scale types, since the classes of meaningful transformations narrow from nominal to ratio scale. *Stevens* recommends using certain statistics only for specific scale types: the mean only for interval scale data, for example [417]. Criticism of such a clerical restriction and the scale type hierarchy itself is presented in [439]. In [309], an alternative list of categories including names, ordered grades, ranks, counted fractions, counts, amounts and balances is proposed. Beyond this, *Guttman* argues that instead of restricting data analysis to permissive statistics for a scale, one should rather consider the loss function to be minimized [160]. This argument points to the problem of measurement involving validity and reliability issues.

In essence, *validity* refers to the question whether conclusions based on a measure are actually accurate and whether measurement captures what is intended to be measured. Since abstract concepts have to be translated into a measurable operational definition or a metric, there is plenty of room for mismatch (see [204, 234]). Validity can be judged in terms of freedom from systematic error. While the true scores are often not available, validity is, in general, difficult to assess. Establishing the validity of a measurement involves the three issues of content validity, criterion validity and construct validity (see [409, 258]). If a measurement is not valid it is also not possible to draw valid conclusions from it.

- *Content Validity*: This type of validity refers to the ability of a measurement scale to represent the full range of meanings associated with the empirical phenomenon. Assuming we want to find out which one of two process models is perceived to be more complex: If a questionnaire only were to offer two answers like “model A seems more complex” and “model B seems more complex”, we would have a problem with content validity since the phenomenon of indifference (“both models seem equally complex”) could not be represented.
- *Criterion Validity*: This type of validity points to the pragmatic value of a measurement, i.e. how closely measurement values and real phenomena are connected. Assume we are interested again in the complexity of a process model as it is perceived by modelers. If we choose to consider the number of arcs as a measurement for it, we might encounter the problem that models with the same number of arcs. For example, a sequential model and a model with arbitrary cycles are perceived as having totally different complexity. The number of arcs might, therefore, have a problem with criterion validity related to complexity since the measurement and the real phenomenon might not be closely connected.
- *Construct Validity*: This type of validity refers to theoretical considerations related to the question why a certain construct is appropriate. Consider again the perceived complexity of process models and assume that the randomly assigned ID number of the model exhibits the capability to rank a sample of process models with respect to complexity. Though we might have a certain degree of criterion validity due to the pragmatic ranking capability, we have a problem with construct validity since there is no theoretical explanation available for the connection between ID and perceived complexity.

An alternative classification of measurement validity is proposed in [433].

Reliability refers to the consistency of measurements over entities and time. While a scale can be reliable and not valid, an unreliable measurement cannot be valid. In this way, reliability establishes an upper bound on validity (see [409, p.364]). In the following sections, we will discuss metrics that can be calculated from graphs, software artifacts and business process models. Reliability essentially relates to the question of whether our calculation algorithm works correctly and deterministically. There are hardly content validity problems since all metrics are based on counts related to the models. We will, however, discuss construct validity from a theoretical point of view for each of the metrics that we identify. We will test criterion validity in the following chapter, but first we present results from the neighboring disciplines network analysis and software measurement.

4.2 Metrics in Network Analysis

Network analysis refers to structure theory and related methods in the area of applied graph theory [63, p.1]. It has a long tradition in social sciences as social network analysis (SNA) dating back to the 1930s (see [389, pp.7]), but there are also applications in engineering related to electrical circuits or in natural sciences like epidemiology. Since EPC business process models are a special class of graphs, it is worth investigating whether network analysis techniques might be applicable.

Network analysis deals with three subareas with a focus on either elements, groups or the overall network [63]. *Element-level* analysis deals with quantifying the position of an element within a network. The Google PageRank is an example that assigns an individual web page a rank based on its connections within a network of web pages. *Group-level* analysis refers to sets of elements of a network and their relationships. An application example is the case of describing political decision making with concurrent groups of interest. *Network-level* analysis considers properties of the overall network. Some properties have been studied related to the International Movie Database (IMDb) about what the average distance between any two actors is. For business process models, we are particularly interested in network-level analysis metrics. In this section, we focus on degree, density, distance, centrality and connectivity metrics. For further details on network analysis refer to [389, 64].

A graph $G = (V, E)$ defined by a set of vertices V and edges $E \subseteq V \times V$ is the starting point of network analysis. EPCs as defined in Definition 2.1 are a specific kind of graph with the sets of functions, events and connectors being the vertices (or nodes) and directed edges called arcs. A basic set of network analysis techniques is related to the degree of the vertices of a graph. The *degree* $d(v)$ of a vertex is the number of edges that are connected to it. For a directed graph the degree of a vertex is the sum of its in-degree (the number of incoming arcs) and its out-degree (the number of outgoing arcs). The average degree

$$\bar{d}(G) = \frac{1}{|V|} \sum_{v \in V} d(v)$$

summarizes whether vertices are connected to many or to few other vertices. Since the sum of degrees equals twice the number of arcs, the average degree can also be calculated as

$$\bar{d}(G) = \frac{2 \cdot |E|}{|V|}$$

A frequently used metric in network analysis is *density* (see [389, pp.69]). Density gives the ratio of edges divided by the maximum number of possible edges. The density Δ of a directed graph is calculated as

$$\Delta(G) = \frac{|E|}{|V| \cdot (|V| - 1)}$$

ranging from 0 for an empty graph to 1 for a complete graph (assuming no self-edges from a vertex to itself). Despite its popularity the results of this metric have to be handled with care. This is due to the fact that complete graphs are rare in practice [389, p.70]. For a random graph, it can be expected that the degrees of the vertices are binomially distributed: only every other potential edge would be included yielding a Δ of 0.5. Several researchers observed that natural graphs do not follow such a binomial, but a distribution that can be expressed as a *power law* (see [65] for an overview). This power law defines the distribution over degrees using two parameters $c > 0$ and $\gamma > 0$ such that the expected amount of vertices with degree k is $ck^{-\gamma}$. The constant c is used for scaling in such a way that the sum over k yields either 1 or the number of vertices. An implication of such a power law distribution is that nodes with a small degree are more likely than nodes with a high degree. In some example graphs, γ values are estimated as for the actor collaboration graph ($\gamma \approx 2.3$) and the power grid of the United States of America ($\gamma \approx 4$) [40]. The implication of this finding is twofold. First, if such a power law is present then graphs become less dense with increasing size. Second, and due to the previous fact, density might be only applicable for a comparison of graphs of the same size. For business process models, it can be expected that nodes with a high degree value are scarce. The implications of the power law distribution must, therefore, be kept in mind when interpreting the density metric of process models.

While density captures the overall cohesion of a graph, *centrality* metrics describe to which extent the cohesion is organized around a central point [389, p.89]. Several centrality metrics are based on the distance matrix $\delta(u, v)$ of a graph in which each entry gives the shortest path between vertices u and v , also called geodesic distance. If the graph does not include cycles with negative weight, the distance matrix can be calculated in polynomial time (see [65] for an overview of algorithms). Based on the distances it is easy to determine the average distance

$$\bar{\delta}(G) = \frac{1}{|V| \cdot (|V| - 1)} \sum_{\substack{u, v \in V \\ u \neq v}} \delta(u, v)$$

and the diameter

$$diam(G) = \max\{\delta(u, v) \mid u, v \in V\}$$

of a connected graph¹. The centrality of a graph depends upon the sum of differences between the score of the most central point and any other point. It is the ratio between this sum of differences and the maximum possible sum of differences (see [133]). Centrality scores can be calculated based on degree sum, closeness and betweenness. While the *sum of degrees* of a vertex only uses information of its local importance in a graph, *closeness* considers the sum of distances between vertices. The closeness centrality of a graph is defined as the inverse of the average distance that a vertex has to all other vertices. Given the maximum sum of these differences, one can calculate the closeness centrality for the overall graph (see [133]). *Betweenness* is based on the idea that certain vertices are more important since they connect subparts of the graph. The betweenness proportion of a vertex y for a pair x and z describes the proportion of shortest paths that connect x and z via y . The pair dependency of x and y is the sum of betweenness proportions of y that involves all pairs containing x . The betweenness score of a vertex is, therefore, half the sum of all dependency scores of that vertex. The most central point of a graph can be identified using either degree sums, closeness or betweenness scores. For further details and formulas see [133, 225]. While the diameter appears to be interesting for measuring the length of an EPC process model, the centrality concept is rather difficult to relate to business process modeling concepts. We will, therefore, disregard centrality metrics for measuring complexity of an EPC.

Connectivity is related to questions about how many vertices have to be removed from the graph to make it unconnected. An interesting concept in this context is that of a cut-vertex. A cut-vertex (also called articulation point or separation vertex) is a vertex which, when removed, increases the number of connected components of a graph. The *number of cut-vertices* can be calculated in $O(|V|+|E|)$ based on a depth-first search tree of the graph (see [84, Ch.22] or [62]). It gives important information about how easily the overall graph can be split into its components. For an EPC, the number of cut-vertices might reveal information in how far the model could be understandable in a divide-and-conquer way, i.e. considering components that are connected via cut-vertices in isolation. Depth-first search trees can also be used to decide whether a graph is planar, i.e. if it can be drawn in such a way that edges do not cross [186]. Since process models are frequently drawn as planar graphs we assume that the planarity property has little selective power and, therefore, disregard it for process models.

In summary, degree, density, distance and connectivity metrics seem to be interesting not only in terms of graphs in general, but also for business process models in particular. We will adapt these metrics to EPCs in Section 4.5. For further network and graph analysis techniques we refer to [389, 84, 64].

¹ For a disconnected graph, the calculation has to be restricted to those pairs that do not have an infinite distance.

4.3 Metrics in the Software Engineering Process

In the software engineering process, several metrics are used to provide input for quality assurance (see [59, 413]). The challenge is to establish a relationship between metrics for internal attributes that are easy to measure and external ones. While internal product attributes can be directly calculated for software artifacts, this does not hold for external attributes like maintainability or reliability as well as unknown parameters like total effort and number of defects. Instead of the term error, software engineering used the generic term defect comprising faults and failures (see [193, 124]). Faults are defects that result from human errors that are encoded in the software. The terms fault and error are often used synonymously. Failures are departures of a software system from its required behavior. Several failures might potentially be tracked back to the same fault as a source. Software measurement approaches typically follow a top down design. The classical Goal-Question-Metric (GQM) approach by *Basili et al.* [45, 44] advocates a clear definition of the overall goal for a software design project. Based on this goal, several questions can be derived related to how the goal can be achieved. Respective metrics offer a quantitative answer to the questions. A similar approach is proposed by *Kan* [204] who suggests first to identify a concept of interest and to define it. In a second step an operational definition has to be found that serves as the basis for a measurement related to the concept. This measurement must be checked for validity and reliability with respect to the concept. In the remainder of this section, we will present several metrics for internal attributes related to the structure of a program like size and complexity which are frequently used in software engineering. These metrics essentially capture control flow, data flow and/or data structure. We will then discuss external quality attributes. External quality attributes model aspects like reliability, usability and maintainability that relate them to internal attributes. We also report on empirical results related to validating these metrics.

Before presenting individual metrics, we must discuss the association between a software program and its control flow graph. Since we aim to investigate the potential analogy between software measurement and business process model measurement we consider software programs to be represented as control flow graphs. A control flow graph $G = (V, E)$ can be derived from a program by mapping blocks of sequential code to vertices with one input and one output arc and branching statements like `if` or `while` to vertices with multiple input and output arcs (see [261]). Metrics defined for control flow graphs can be easily adapted to process models.

The *lines of code* metric is the traditional metric for measuring the size of software. While the idea of counting the lines of a software program is rather simple, there have been several discussions regarding whether comments should be included. *Jones* states that depending on the way lines are counted, the results might differ by the factor of five [201]. Standardization efforts such as the 242 pages long report of the Software Engineering Institute of Carnegie Mellon University illustrate the extent of choices in this area [391]. Given the control flow graph $G = (V, E)$ of a program, the lines of code can be identified with the number of vertices:

$$LoC(G) = |V|$$

One problem with lines of code is that code from different programming languages is not directly comparable. Research on language productivity has established so-called gearing factors that capture the effort of writing an arbitrary program in a certain language. For example, programming in SQL requires an average effort of 39 compared to 172 in Assembler [234, p.38]. An increase in lines of code does not necessarily imply an increase in functionality. Function Point Analysis, originally proposed in [22], addresses this problem by assigning a score to each input, output, interface, data file and inquiries based on their individual difficulty. While function points can be compared for programs written in different languages, there is the problem that function point analysis requires human interpretation of program difficulty. It can, therefore, be only partially automated. This is a major disadvantage compared to lines of code that can be counted automatically.

The *cyclomatic number* CC proposed in [260, 261, 262] is an early complexity metric. It is based on the control flow graph $G = (V, E)$ of a program and captures the number of paths through a program that are needed to visit all vertices. More precisely, it matches the maximum number of linearly independent paths through a program. The cyclomatic number is of particular importance for test theory since it defines the number of test cases required for unit tests. It can either be calculated based on the number of vertices and edges as

$$CC(G) = |E| - |V| + 1$$

or alternatively by counting the number of choices weighted with the number of alternatives. Since CC is not biased towards a specific programming language, it can be used to compare the complexity of programs written in different languages. The cyclomatic complexity density CCD is an CC extension for predicting maintenance productivity [142]. CCD is calculated as CC divided by the lines of code LoC . A second extension called essential cyclomatic complexity ECC is a measure of unstructuredness of the program [260]. It is calculated as the cyclomatic number of the code after removing all structured constructs like `if` and `while` statements. A totally structured program, therefore, has an essential cyclomatic complexity of zero. The cyclomatic number can be calculated for EPC business process models that do not include concurrency (see Figure 4.1).

Halstead's metrics provide measurable definitions for the length, vocabulary, volume, difficulty and effort of a software program based on the number of operators and operands [162]. Operators comprise commands and structuring elements like parentheses. Operands basically refer to elements that have a value like variables and constants. The operator parameters n_1 and N_1 refer to the number of distinct operators and the total occurrences of operators, respectively. The operand parameters n_2 and N_2 describe the number of distinct operands and the total occurrences of operands, respectively. *Halstead* then defines the following metrics:

$$\text{Length } N = N_1 + N_2$$

$$\text{Vocabulary } n = n_1 + n_2$$

$$\text{Volume } V = N \cdot \log_2(n)$$

$$\text{Difficulty } D = n_1/2 \cdot N_2/n_2$$

$$\text{Effort } E = D \cdot V$$

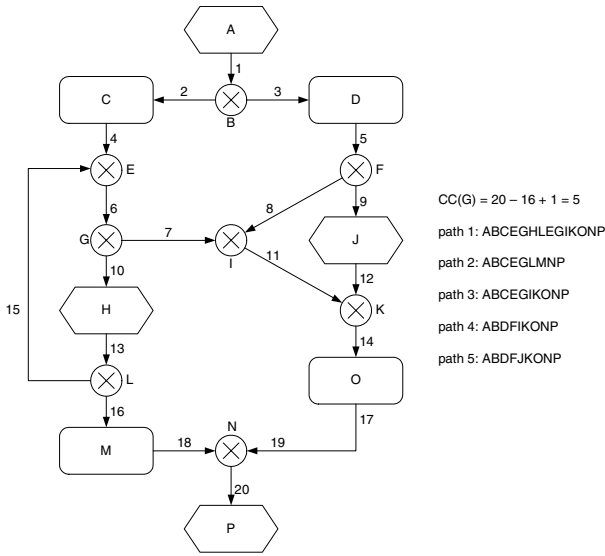


Figure 4.1. Cyclomatic number of an EPC without concurrency

The formula of volume is motivated by the number of binary mental comparisons needed to write a program of length N . The formula for difficulty is a postulate of *Halstead's* theory. *Halstead* claims with reference to the psychologist *Stroud* that the required programming time T can be calculated as $T = E/18sec$. Although the work by *Halstead* had a lasting impact on software metrics research, it has been criticized repeatedly as “an example of confused and inadequate measurement” [124].

The *information flow metric* proposed by *Henry and Kafura* [177] is based on the data flow between different modules of a program. Information can be passed between modules in three ways: by a module invoking another one; by returning a result from the invoked module to the caller; and by exchanging information via a global data structure. The *fan-in* of a module M refers to the number of calls to the module plus the number of reads to global data structures. The *fan-out* captures calls to other modules and write operations to global data structures. Based on these parameters, the information flow complexity is calculated as

$$IFC(M) = LoC(M) \cdot (fan_{in} \cdot fan_{out})^2$$

The *IFC* metric can be used at design-time to predict which modules are likely to contain errors. The multiplication of fan-in and fan-out has been criticized in [124]. If a module has either no fan-in or fan-out the metric yields zero which is misleading.

The lines of code, cyclomatic complexity, *Halstead* complexity and information flow complexity metrics were developed for procedural programming languages. In [81], a set of metrics for object-oriented development is proposed. In this context, coupling and cohesion are important concepts. Coupling describes the degree of interdependence between a pair of modules and cohesion describes the degree to which

the elements of a module are functional related (see [468, 413]). The proposal is based on ontological work of *Bunge, Wand and Weber* [66, 448] and includes six metrics.

- *Weighted methods per class*: This metric is calculated as the sum of complexity weights over all methods.
- *Depth of inheritance tree*: The inheritance tree defines a hierarchy of classes from which an individual class inherits properties and methods. The metric gives the depth of such inheritance for a class in order to describe how many ancestors potentially affect it.
- *Number of children*: This metric gives the number of successors in the inheritance tree for a particular class.
- *Coupling between object classes*: This measure states to how many other classes a class is coupled.
- *Response for class*: This metric gives the size of the set of methods that a class might use to respond to a message. It is calculated as the sum of local methods plus methods called by these local methods.
- *Lack of cohesion metric*: This is the number of disjoint sets identified by comparing each pair of methods and the instance variables they relate to. A high value for this metric would suggest a split up of the class.

A generic framework for classifying further object-oriented metrics is proposed and validated against 350 metrics in [435].

The previously presented metrics related to internal attributes of a software artifact were tested in various empirical studies for their criterion validity with respect to predicting defects or project effort. *Fenton and Ohlsson* test several hypotheses on faults and failures that *LoC* is a useful predictor for defects and that complexity metrics perform better [123]. They find that *LoC* is indeed suited to rank modules according to their absolute number of faults. They also confirm the finding of *Basili and Perricone* [43] that fault density seems to decrease with module size. Cyclomatic complexity *CC* does not seem to outperform *LoC* as a predictor for errors. *Fenton and Ohlsson* propose to rather consider *CC* as a predictor for comprehensibility and maintainability.

An overview of effort estimation is given in [234, pp.79]. Benchmark data is useful to assess productivity in particular. For example, in one staff month, code produced for data processing applications appear to range between 165 and 500 lines [234, p.86]. Measurements are also used to predict the dynamics of effort and defects throughout the development process. Given a record of found defects and a typical distribution of defects over time, one can estimate defects to be found in future. A typical distribution in this context is the Rayleigh curve that sharply steepens to a maximum and then decreasing falls towards zero. Such an estimation is particularly helpful for determining a suitable release date (see [234]). Regression-based prediction can also be applied in terms of effort. *Boehm's* constructive cost model (COCOMO) assumes that effort $E = aS^bF$ with S measuring size, F an adjustment factor, and a and b depending on the type of software being constructed [60]. The ISO/IEC 9126 Software Product Quality Model [195] provides an over-

all classification scheme for quality-related product attributes on four levels. It can be used as a framework for several metrics defined earlier. Further models and empirical results are summarized in various text books on software measurement (see [474, 124, 204, 234]).

4.4 Related Work on Metrics for Process Models

In this section, we discuss related work on metrics for process models. Since the authors hardly refer to each other, it is difficult to group the contributions according to subject area. We will, therefore, present their work mainly chronologically.

In 1990, *Lee and Yoon* conducted pioneering work on the definition of metrics for Petri net process models and their empirical validation [243, 244]. The authors group their metrics into two categories: *structural metrics* include simple counts of places, transitions, arcs and the cyclomatic number of the control graph. *Dynamic metrics* cover the number of markings and the maximum and average number of tokens for the original and a reduced state space. In an empirical study of a set of 75 Petri nets, the authors find that a simple adaptation of the cyclomatic number is not a suitable complexity metric; that the number of places can serve as a predictor for the size of the reachability graph in an exponential equation; and that reduction techniques reduce the analysis problem by factor four for the sample nets.

Nissen was among the first to introduce measurement concepts for business process modeling and business process design in particular (see [318, 319, 320]). His work is motivated by the observation that business process design is an ill-structured process and that it is, therefore, a case for business process reengineering on itself. Based on a set of measurements for process models, he utilizes design heuristics and a knowledge-based system for reasoning on the quality of the design. The proposed metrics cover counts for *distinct paths*, *hierarchy levels*, *nodes* in the process model, *cycles*, *diameter* and *parallelism* as number of nodes divided by the diameter. These metrics are provided by the KOPeR tool which guides the process reengineering process [320].

Tjaden, Narasmihan and Gupta operationalize four characteristics of a business process that need to be balanced: simplicity, flexibility, integration and efficiency [430, 429]. *Simplicity* is calculated based on so-called basic complexity of the process, i.e. in essence the sum of nodes, arcs and roles. The overall simplicity is then formalized as the average activity complexity divided by the maximum activity complexity. *Flexibility* and *Integration* are determined based on a list of scores similar to function point analysis. Though it is mentioned, the technical report [429] does not operationalize efficiency. *Balasubramanian and Gupta* criticize the high level of abstraction of the metrics by Tjaden (see [37]). A function point approach, such as it is proposed for flexibility and integration, is difficult to automate.

Building on measurement efforts for concurrent programs (see [396]), *Morasca* proposes a set of metrics for Petri nets and a theoretical foundation [308]. He identifies size, length, structural complexity and coupling as interesting attributes of a Petri net and for each attribute he defines a set of axiomatic properties which a respective

metric would have to fulfill. For size he proposes *number of places and transitions*, for length the maximum minimal distance (i.e. the *diameter*), for structural complexity the *number of base paths*, the *concurrent contribution* as the number of arcs minus two times the number of transitions, the *sequential contribution* as the number of transitions with one input places and for coupling the *number of inbound or outbound arcs of a subnet*. Since *Morasca's* contribution is theoretical, an empirical validation is not included in his paper.

Latva-Koivisto [239] proposes several complexity metrics for business process models including the *Coefficient of Network Connectivity*, *Cyclomatic Number*, *Reduction Complexity Index*, *Restrictiveness Estimator* and *Number of Trees in a Graph* as metrics. This work has two basic weaknesses. First, there is no distinction made between different kinds of routing elements and second a motivation to consider the restrictiveness estimator and the number of trees in a graph as a complexity metric is missing. The work by *Latva-Koivisto* did not receive much attention in the community since it was published only as a technical report.

A different stream of research adapts coupling and cohesion concepts from software engineering (see [397, 68, 393, 57]). In the tradition of this work, *Daneva et al.* [94] introduce a set of complexity metrics for EPCs based on the notational elements of the model: *function cohesion*, *event cohesion* and *cohesion of a logical connector*. From a limited validation with 11 EPCs they conclude that these metrics help to identify error-prone model fragments. In [347, 348], *Reijers and Vanderfeesten* develop a set of coupling and cohesion metrics to guide the design of workflow processes. The coupling of an activity is calculated based on its relation cohesion and its information cohesion. The *activity relation cohesion* λ describes in how far an activity shares control flow input and output with other activities. It is defined as the sum of overlaps between each activity pair divided by the maximum number of pairs with choices not being considered as overlap. The *activity information cohesion* μ determines how many information objects are used by an activity more than once in relation to all information objects in use. The cohesion of an activity is calculated as $\lambda \cdot \mu$. The overall *process cohesion* c is then defined as the average activity cohesion. The *process coupling* k metric is calculated as the number of connected activities divided by the maximum number of connections. That is, process coupling is equated with the density Δ of the process graph. Inspired by [394], the authors define a process coupling/cohesion ratio as $\rho = k/c$ with a low value indicating good design.

The work of *Cardoso* is centered around an adaptation of the cyclomatic number for business processes he calls *control-flow complexity (CFC)* [69, 72]. *CFC* is calculated from a model by summing up the split connectors weighted by the combinations of output markings they can produce: 1 for an AND-split S_{and} , $|S_{xor} \bullet|$ for an XOR-split S_{xor} , and $2^{|S_{or} \bullet|} - 1$ for an OR-split. The *CFC* metric was validated against a set of complexity axioms proposed by *Weyuker* [454] in [70, 71] and tested with respect to their correlation with perceived complexity [73]. In an analysis of the SAP Reference Model, however, the *CFC* metric was found unsuitable for the prediction of errors in business process models [276].

The research conducted by a group including *Canfora, Rolón, García, Piattini, Ruiz and Visaggio* extends work related to measurement of the software process. In [67], *Canfora et al.* present a set of metrics and evaluate their suitability to serve as predictors for maintainability of the process model. The operational definition of maintainability covers *analyzability* as the likeliness of discovering errors or deficiencies in the model, *understandability* as the likeliness of comprehending a model and *modifiability* as the likeliness of correctly changing the structure of the model. The metrics address structural properties of the process model related to size, complexity and coupling. For *size* they include count metrics based on the number of activities, work products, process roles, input and output dependencies of work products with activities, ratio of work products to activities and ratio of process roles to activities. For *complexity* they consider number of dependencies between work products and activities and ratio between input or output dependencies of work products with activities and total number of dependencies. *Coupling* is measured as number of precedence dependencies between activities and activity coupling. In a set of related experiments, the authors find that most of the metrics are correlated to maintainability: the numbers of activities, work products, work product dependencies and precedence dependencies show good results while activities coupling and the ratio of roles to activities are only correlated to a limited extent. The number of roles and the ratios related to work product dependencies are not correlated. *Rolón et al.* extend this set of metrics and tailor it to the specifics of BPMN reflecting additional numbers for events, gateways, message flows and pools [354, 355, 356, 353].

Inspired by *Nissen's* and *Tjaden's* work, *Balasubramanian and Gupta* propose a set of metrics to support business process design (see [37]). This set includes, among others, metrics to quantify the degree of automatic decision making (*branching automation*), *activity automation*, *role integration*, *role dependency*, *activity parallelism* and *transition delay risk*. These metrics are applied in a case study for identifying the best of two alternative process designs.

Cognitive considerations play an important role for understanding good design in software engineering, for example [58]. This is explicitly reflected in some approaches to metrics. The work by *Gruhn and Laue* adapts cognitive weights from software engineering to business process models [152, 153]. The approach by *Vanderfeesten et al.* defines the cross-connectivity metric. Its aim is to capture how difficult it is to understand how two nodes in a process model relate to each other [437]. This metric is validated for predicting errors and understanding of process models.

Only partially related is the work by *Etien and Rolland* on measuring the fitness of a business process model with a real-world process [121]. The authors define several metrics related to concepts of the Bunge-Wand-Weber ontology which express in how far certain concepts are represented in a model that is present in the real world. In practice, such a measurement maps to a comparison of a business process model with a potentially more complete model that captures entities of the BWW-ontology related to the real-world process. Such a measurement is accordingly more of theoretical interest than of benefit in the design process.

Modularity is another concept that builds on insights on software design. In essence, modularity relates to the number, the size and the depth of nesting of mod-

ules. The approach by *Lassen and Van der Aalst* identifies structured components in arbitrary process models for the translation to structured BPEL [14, 17]. These components can also be used to describe the process model in a quantitative way. The work by *Vanhatalo et al.* builds on the program structure tree concept for formal computer program analysis [438]. Using a linear time algorithm, a program structure tree can be derived that includes a nesting of all single-entry-single-exit components of the process model. The authors use maximum, average and minimum component size to describe their model sample. The structuredness of the process model can be defined depending on the structure of the components [242].

The two survey papers by *Cardoso, Mendling, Neumann and Reijers* [74] and *Gruhn and Laue* [241] summarize a great share of the earlier mentioned work. They reveal that several metrics for business process models are adaptations of software complexity metrics. This also holds true for an adaptation of complexity metrics for EPCs as reported in [155]. Only some authors provide operationalizations for central concepts of business process models such as parallelism, cycles and sequentiality (see [320]). Several process-related aspects like structuredness and mismatch of connectors are covered only to a limited extent or not at all. In the following section, we will identify several metrics that are specifically tailored for predicting errors in business process models.

4.5 Definition of Metrics for Process Models

In the previous sections we have discussed several metrics for business process models. Many of them aim to operationalize the structural complexity of the process graph. The term complexity is discussed in software measurement and authors try to identify axioms that a complexity metric would have to fulfill (see the work by *Weyuker* [454]). Such an axiomatic approach is criticized from different perspectives. In [79], the authors show that the *Weyuker* axioms do not guarantee that the metric is meaningful by defining an obviously useless metric that meets the axioms. More serious is the criticism by *Zuse* who shows that the aspects subsumed under the term complexity cannot be captured by a single metric alone [474]. *Fenton and Pfleeger* reinforce this finding by pointing to measurement theory [124, pp.322], saying that every valid measurement should obey the rules of representational theory of measurement. This implies that first an empirical concept or relationship in the real world should be identified, followed by suitable measurements. In other words, the concept guides the measurement and the metric does not define the concept. This principle is incorporated in several software measurement approaches such as the Goal Question Metric (GQM) approach by *Basili et al.* [45, 44] or the Concept, Definition, Operational Definition, Measurement approach by *Kan* [204].

Following this line of argument, we consider the comprehensibility of the business process model as the main determinant for error probability. This is based on the assumption that process models are constructed by human modelers and that their design is subject to bounded rationality [404]. The comprehensibility of any model by a person is influenced by a variety of factors including *model-related* factors like

size, *personal factors* like modeling competence, *purpose* of modeling like documentation or execution, *domain knowledge*, *modeling language* or *graphical layout* of the model. In this chapter, we only investigate the model-related aspect. More precisely, we analyze several metrics that capture various aspects related to either the *process model structure*, the *process model state space*, or both (see [244]) and discuss their impact on error probability. Each metric is presented by giving its (1) symbol, (2) definition, (3) rationale why it should be considered, (4) limitations, (5) the hypothesis related to it and (6) related work that mentions similar metrics in previous research. We consider a business process model to be a special kind of graph $G = (N, A)$ with at least three node types $N = T \cup S \cup J$, that is *tasks* T , *splits* S , *joins* J and *control flow arcs* $A \subseteq N \times N$ to connect them. We use the generic term connectors $C = S \cup J$ for splits and joins collectively. Each connector has a *label* AND, OR or XOR that gives its routing of merging semantics. For presentation purposes we subdivide the set of metrics into the categories *size*, *density*, *partitionability*, *connector interplay*, *cyclicity* and *concurrency*.

4.5.1 Size

Several papers point to *size* as an important factor for the comprehensibility of software and process models (see [324, 67, 74, 241, 474, 124, 204]). While the size of software is frequently equated with lines of code, the size of a process model is often related to the number of nodes N of the process model. Furthermore, we consider the *diameter* of the process graph.

Symbol	S_N
Definition	Number of nodes of the process model graph G
Metric	$S_N(G) = N $
Rationale	A larger business process model in terms of $S_N(G)$ should be more likely to contain errors than a small one since the modeler would only be able to perceive a certain amount of nodes in a certain period of time.
Limitations	There are obviously large models in terms of $S_N(G)$ that are unlikely to have errors, e.g. if the model is sequential without any connectors.
Hypothesis	An increase in $S_N(G)$ should imply an increase in error probability of the overall model.
Related Work	<i>LoC</i> [474, 124, 204]; Number of nodes [244, 320, 308, 37, 67, 74, 241, 356].

The size metric S_N does not differentiate between the several node types and its subsets of an EPC. We define size metrics for each EPC element type and its subsets by mentioning it as the index of S , i.e. S_{E_E} refers to the number of end events of an EPC, and S_F to the number of functions of an EPC. The size of the model might be closer related to the longest path of it and we, therefore, define the diameter *diam* of a process model.

Symbol	<i>diam</i>
Definition	The diameter gives the length of the longest path from a start node to an end node in the process model.
Metric	$diam(G)$ is calculated either based on the distance matrix of a process model or based on shortest path algorithms (see [65])
Rationale	A larger business process model in terms of $diam(G)$ should be more likely to contain errors than a small one since the modeler would only be able to perceive a certain amount of consecutive nodes in a certain period of time.
Limitations	There are obviously large models in terms of $diam(G)$ that are unlikely to have errors, for example if the model is sequential.
Hypothesis	An increase in $diam(G)$ should imply an increase in error probability of the overall model.
Related Work	The diameter was also proposed in [320, 308].

Figure 4.2 illustrates one particular shortcoming of size as a simple count metric. The left and the right model have exactly the same number of nodes but the diameter of the right EPC is longer. This difference between the models does not become apparent if only the size metric S_N is considered.

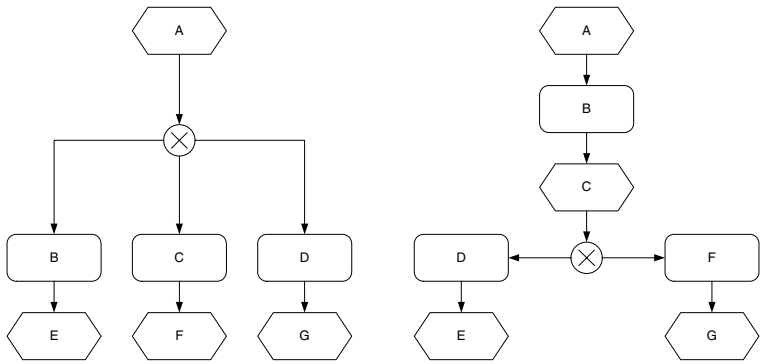


Figure 4.2. Two EPCs of the same size with different diameter

4.5.2 Density

In the context of this book we use density as a generic term to refer to any metric that relates numbers of nodes to numbers of arcs. There are several metrics that provide information about the relation of arcs and nodes. Here we consider the *density* metric, the *coefficient of network connectivity*, *average connector degree* and *maximum degree*.

Symbol	Δ
Definition	The density of the process graph refers to the number of arcs divided by the number of the maximum number of arcs for the same number of nodes.
Metric	$\Delta(G) = \frac{ A }{ N \cdot (N - 1)}$
Rationale	A business process model with a high density $\Delta(G)$ should be more likely to contain errors than a less dense model with the same number of nodes.
Limitations	The density in terms of Δ is difficult to compare for models with a different number of nodes: larger models with the same average degree have a smaller density since the maximum possible number of arcs grows by the square of $ N $. Many natural graphs seem to obey a power law (see [64]) which implies that larger models would be less dense in terms of Δ .
Hypothesis	An increase in $\Delta(G)$ should imply an increase in error probability of the overall model.
Related Work	Δ is mentioned as process coupling metric in [348].

Symbol	CNC
Definition	The coefficient of connectivity gives the ratio of arcs to nodes.
Metric	$CNC(G) = \frac{ A }{ N }$
Rationale	A denser business process model in terms of $CNC(G)$ should be more likely to contain errors since the modeler has to perceive more connections between nodes than in a model that is less dense.
Limitations	There are process models with the same CNC value that might differ in error probability. Consider, for example, a sequential model without any connector and a model with the same number of nodes having one split-connector. The two models also have the same number of arcs, therefore CNC is equivalent.
Hypothesis	An increase in $CNC(G)$ should imply an increase in error probability of the overall model.
Related Work	CNC is listed in [239, 74]. The inverse of CNC called activity coupling $NCA = N / A = 1/CNC$ is proposed in [67, 356].

Symbol	$\overline{d_C}$
Definition	The average degree of connectors gives the number of nodes a connector is in average connected to.
Metric	$\overline{d_C}(G) = \frac{1}{ C } \sum_{c \in C} d(c)$

Rationale	A denser business process model in terms of $\overline{d_C}(G)$ should be more likely to contain errors since the modeler has to perceive more connections between nodes than in a model that is less dense.
Limitations	There are process models with the same $\overline{d_C}$ value which differ in size. They should, therefore, also differ in error probability.
Hypothesis	An increase in $\overline{d_C}(G)$ should imply an increase in error probability of the overall model.
Related Work	$\overline{d_C}$ is related to the information flow metric by <i>Henry and Kafura</i> [177] and its adaptation to process models in [74, 241].

Symbol	$\widehat{d_C}$
Definition	The maximum degree of a connector.
Metric	$\widehat{d_C}(G) = \max\{d(c) \mid c \in C\}$
Rationale	A business process model with a high maximum degree $\widehat{d_C}(G)$ should be more likely to contain errors since the modeler has to perceive more connections between the connector of maximum degree than in a model that has a lower maximum degree.
Limitations	There are obviously process models with a high maximum degree and the same $\widehat{d_C}$, but with a low average degree.
Hypothesis	An increase in $\widehat{d_C}(G)$ should imply an increase in error probability of the overall model.
Related Work	$\widehat{d_C}$ is closely related to the information flow metric by <i>Henry and Kafura</i> [177]. The idea of information flow is to identify modules whose interactions are difficult to comprehend and the connector of maximum degree is the most difficult following this line of argumentation.

Figure 4.3 illustrates the sensitivity of the different density metrics to size. The EPC on the left-hand side has 18 nodes and 17 arcs while the one on the right-hand side only has 8 nodes and 7 arcs. The density metric Δ of the smaller model is more than twice as high (0.055 to 0.125) than that of the larger one although the structure is quite similar. This fact reinforces the statements of Section 4.2 on the difficulty to compare graphs of different size by the density value. The *CNC* metric reflects the similar structure much better with similar values (0.945 to 0.875). The average and the maximum degree of connectors $\overline{d_C}$ and $\widehat{d_C}$ yield 3 for both models. In this case, they underline the similarity of the models well.

4.5.3 Partitionability

We use the term partitionability for referring to those aspects of a process model that relate to the relationship of subcomponents to the overall model. We discuss *separability* and *sequentiality* in particular, both of which capture how far certain parts of the model can be considered in isolation. Furthermore, *structuredness* quantifies

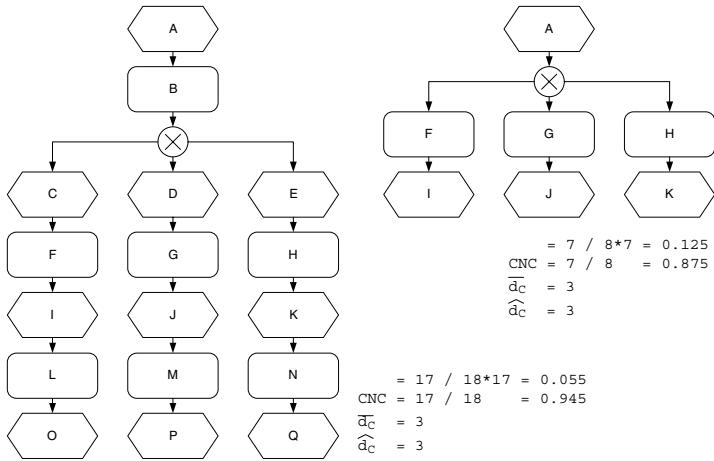


Figure 4.3. Two EPCs of the same average and maximum connector degree and varying density and CNC values

to which degree components are composed in a structured way while *depth* defines how far a certain node is from a start or end event.

Separability is closely related to the notion of a cut-vertex (or articulation point), i.e. a node whose deletion separates the process model into multiple components. We define the separability ratio as the number of cut-vertices to number of nodes. Cut-vertices can be found using depth-first search.

Symbol	Π
Definition	The separability ratio relates the number of cut-vertices to the number of nodes.
Metric	$\Pi(G) = \frac{ \{n \in N \mid n \text{ is cut-vertex}\} }{ N - 2}$
Rationale	A model with a high ratio of cut-vertices should be less likely to contain errors than a model with a low ratio. If every node except the start and the end node is a cut-vertex, the model is sequential and should, thus, be easy to understand.
Limitations	The separability ratio Π can be low if there are two long sequential paths in parallel since none of the parallel nodes is a cut-vertex.
Hypothesis	An increase in $\Pi(G)$ should imply a decrease in error probability of the overall model.
Related Work	Π has not yet been considered as a business process model metric.

Sequentiality relates to the fact that sequences of consecutive tasks are the most simple components of a process model. The sequentiality ratio relates arcs of a sequence to the total number of arcs.

Symbol	Ξ
Definition	The sequentiality ratio is the number of arcs between non-connector nodes divided by the number of arcs.
Metric	$\Xi(G) = \frac{ A \cap (T \times T) }{ A }$
Rationale	A process model with a high sequentiality ratio should be less likely to contain errors than one with a low sequentiality ratio. In contrast to the separability ratio Π , the sequentiality ratio Ξ also considers sequences that are in parallel or exclusive. If every arc connects only non-connector nodes, the model is sequential and the sequentiality ratio is 1.
Limitations	There are models with the same sequentiality ratio but whose non-sequential arcs might differ in their degree of comprehensibility.
Hypothesis	An increase in $\Xi(G)$ should imply a decrease in error probability of the overall model.
Related Work	Ξ has not yet been considered as a business process model metric. It is related to sequential contribution of <i>Morasca</i> [308].

Figure 4.4 illustrates the difference between the separability and the sequentiality ratio. The model on the left-hand side includes two cut vertices (the XOR-split and the XOR-join) while the model on the right-hand side additionally has the cut vertices B, C, F and G . Even though the number of nodes is the same in both models, the separability ratio is 0.25 for the left model and 0.75 for the right model. The sequentiality ratio counts sequence arcs no matter in which part of the EPC they appear. The sequential components B to F and C to G , therefore, contribute as much to the sequentiality ratio of the left model as the sequences A to C and F to H in the right model (40%).

Structuredness relates to how far a process model can be built by nesting blocks of matching join and split connectors (see [215]). The degree of structuredness can be determined by applying reduction rules and comparing the size of the reduced model to the original size. We consider the reduction of trivial constructs, structured blocks and loops and of structured start and end components as defined in Section 3.3.2.

Symbol	Φ
Definition	The structuredness ratio of the process graph is one minus the number of nodes in the reduced process graph G' divided by the number of nodes in the original process graph G .
Metric	$\Phi_N = 1 - \frac{S_N(G')}{S_N(G)}$
Rationale	A process model with a high structuredness ratio should be more likely to contain errors than one with a low ratio. If every node is part of a structured block, the model is structured and the structuredness ratio is 1.
Limitations	There are models with the same structuredness ratio but which differ in their degree of comprehensibility, e.g. if one is larger.

Hypothesis	An increase in $\Phi(G)$ should imply a decrease in error probability of the overall model.
Related Work	Φ has not yet been formalized in literature but mentioned in [241, 154]. It is related to essential cyclomatic complexity [260].

Figure 4.5 illustrates the structuredness ratio by the help of two EPCs. The EPC on the left-hand side is totally structured and yields a structuredness value² of 1. The EPC on the right-hand side contains one additional arc and two XOR-connectors that affect the structuredness. While the nodes B to G are deleted by the reduction rule for trivial constructs, the connectors cannot be eliminated. Only 6 out of 12 nodes are deleted yielding a structuredness ratio of 0.5. In the EPC on the right-hand side, the arc between the left and the right column blocks the application of further reduction rules.

Depth relates to the maximum nesting of structured blocks in a process. To calculate depth also for unstructured models we define an algorithm that calculates the in-depth $\lambda_{in}(n)$ of a node n relative to its predecessor nodes $\bullet n$. All nodes are initialized with an in-depth value of 0. The process model is then traversed along all paths starting from a start node and ending with either an end node or a node that was visited before in this path.³ At each visited node n , the new in-depth value $\lambda'_{in}(n)$ is updated based on the value of the previously visited predecessor node $\lambda_{in}(pre)$ and the current value $\lambda_{in}(n)$ according to the following rule:

$$\lambda'_{in}(n) = \begin{cases} \max(\lambda_{in}(n), \lambda_{in}(pre) + 1) & \text{if } pre \in S \wedge n \notin J \\ \max(\lambda_{in}(n), \lambda_{in}(pre)) & \text{if } pre \in S \wedge n \in J \\ \max(\lambda_{in}(n), \lambda_{in}(pre)) & \text{if } pre \notin S \wedge n \notin J \\ \max(\lambda_{in}(n), \lambda_{in}(pre) - 1) & \text{if } pre_i \notin S \wedge n \in J \end{cases}$$

This definition of in-depth captures the maximum number of split connectors that have to be visited to arrive at a node minus the number of joins on this path. The out-depth $\lambda_{out}(n)$ is defined analogously with respect to the successor nodes and decreased for splits and increased for joins. In a structured model, $\lambda_{in}(n)$ equates with $\lambda_{out}(n)$. We define depth $\lambda(n)$ as the minimum of in-depth $\lambda_{in}(n)$ and out-depth $\lambda_{out}(n)$. The depth of the process model A is then the maximum depth $\lambda(n)$ over n .

Symbol	A
Definition	The depth is the maximum depth of all nodes.
Metric	$\lambda(G) = \max\{\lambda(n) \mid n \in N\}$
Rationale	A process model with a high depth should be more likely to contain errors since connectors are deeply nested.
Limitations	There are models with the same depth that differ in size.

² Please note that if the reduced EPC has the minimum size of 2, i.e. if only one start and one end event are connected by an arc, then the value of Φ is set to 1.

³ This definition addresses potential problems with the existence of a fixed point by visiting each node only once for each path even if it is on a loop.

Hypothesis An increase in $\Lambda(G)$ should imply an increase in error probability of the overall model.

Related Work Λ has not yet been formalized but mentioned in [320, 241].

Figure 4.6 shows a structured EPC with its in-depth and out-depth values next to the nodes. It can be seen that both in-depth and out-depth increase with each visit to an XOR-split while visiting an XOR-join decreases both parameters. It must be mentioned that joins are used as entries to loops and splits as exits. The depth calculation algorithm is not able to identify such loops as deeper nested structures. Still, it offers a way to quantify the depth of any process model, no matter if it is structured or not.

4.5.4 Connector Interplay

In this section, we present metrics related to connectors and their interplay. In particular, we discuss connector mismatch, connector heterogeneity and the control flow complexity metric.

Structuredness implies that each split-connector matches a corresponding join of the same type. A mismatch might be the source of an error. Depending on the connector label and the degree, we define the connector mismatch as $MM_l = |\sum_{c \in S_l} d(c) - \sum_{c \in J_l} d(c)|$ where l is the connector type. The mismatch ratio MM gives the sum of mismatch for each connector type.

Symbol	MM
Definition	The connector mismatch gives the sum of mismatches for each connector type.
Metric	$MM(G) = MM_{or} + MM_{xor} + MM_{and}$
Rationale	A process model with a high mismatch is likely to include errors since parallel tokens might not be synchronized or alternative branches might run into AND-joins and deadlock. If the model is structured with matching split- and join connectors MM is zero.
Limitations	Languages like EPCs offer multiple start and end events. They, therefore, might show a mismatch without having errors.
Hypothesis	An increase in $MM(G)$ should imply an increase in error probability of the overall model.
Related Work	MM has not yet been formalized in literature.

Connector heterogeneity refers to which extent different connectors are used in a business process model. For defining a suitable metric that ranges from 0 in the case that there are only connectors of one type, to 1 in the case that there are the same amount of connectors of all three types, we refer to the information entropy measure which has exactly these characteristics. In contrast to the original work of *Shannon and Weaver* [395], we do not consider a binary encoding but a ternary because of the three connector types. The base of the logarithm is therefore three, not two. We also utilize the relative frequency $p(l) = |C_l|/|C|$. The connector heterogeneity is

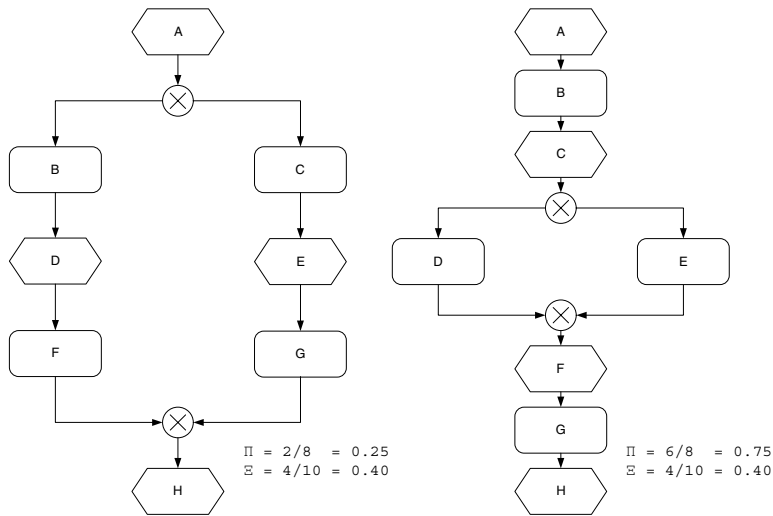


Figure 4.4. Two EPCs with the same sequentiality and different separability

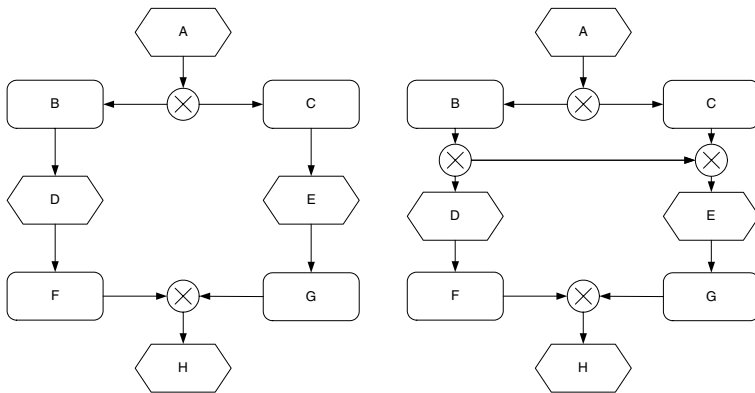


Figure 4.5. Two EPCs with the same functions and events but different degree of structuredness

then calculated analogously to information entropy as the negative sum over the three connector types of $p(l) \cdot \log_3(p(l))$.

Symbol	CH
Definition	The connector heterogeneity gives the entropy over the different connector types.
Metric	$CH(G) = - \sum_{l \in \{and, xor, or\}} p(l) \cdot \log_3(p(l))$
Rationale	A process model with a high heterogeneity is likely to include errors since connectors can be mismatched more easily. If the model includes only one connector type then CH is 0.

Limitations	Process models might have a high connector heterogeneity but if the model is structured an error is less likely.
Hypothesis	An increase in $CH(G)$ should imply an increase in error probability of the overall model.
Related Work	CH has not yet been formalized in literature.

The Control Flow Complexity metric was introduced in [72] for measuring how difficult it is to consider all potential states after a split depending on its type.

Symbol	CFC
Definition	CFC is the sum over all connectors weighted by their potential combinations of states after the split.
Metric	$CFC(G) = \sum_{c \in S_{and}} 1 + \sum_{c \in S_{xor}} c_{xor} \bullet + \sum_{c \in S_{or}} 2^{ c_{or} \bullet } - 1$
Rationale	A process model with a high CFC should be more likely to contain errors according to the above argument.
Limitations	Models with the same structure but with different connector labels may have a huge difference in CFC while they are equally easy to understand.
Hypothesis	An increase in $CFC(G)$ should imply an increase in error probability of the overall model.
Related Work	CFC is inspired by [260] and introduced in [72].

Figure 4.7 illustrates the three connector metrics by the help of two example EPCs. The EPC on the left-hand side has six XOR-connectors, each with a matching counterpart. The mismatch value is 0. Since there are only XOR-connectors, the heterogeneity value is also 0. The CFC value is 6 since each of the three split connectors represents a binary choice. The EPC on the right-hand side has the same structure but partially different connector types. There is one AND-split and an OR-join that do not have a matching counterpart. This results in a mismatch value of 4. Three OR, two XOR and one AND-connector yield a high heterogeneity value of 0.92. Finally, the CFC value is calculated by summing up 2 for the XOR-split and 3 for the OR-split which gives a result of 5.

4.5.5 Cyclicity

Cyclic parts of a model are presumably more difficult to understand than sequential parts. $|N_C|$ gives the number of nodes n_i for which a cycle exists such that $n_i \hookrightarrow n_i$, and cyclicity relates it to the total number of nodes.

Symbol	CYC
Definition	Cyclicity relates nodes on a cycle to the number of nodes.
Metric	$CYC_N = N_C / N $
Rationale	A process model with a high cyclicity should be more likely to contain errors. For a sequential model cyclicity is 0.

Limitations	There are models with the same cyclicity but which differ in comprehensibility, e.g. if one is larger.
Hypothesis	An increase in $CYC(G)$ should imply an increase in error probability of the overall model.
Related Work	CYC has not yet been mentioned in literature. <i>Nissen</i> proposes to count the number of cycles instead [320].

Figure 4.8 depicts two similar EPCs. They have the same cyclicity since they have the same number of nodes that are on a loop. The difference is that the model on the right-hand side has two cycles that are nested in another cycle while the EPC on the left side has two structured XOR-blocks on one loop. Since the cyclicity metric only captures how many nodes are on a cycle it cannot distinguish between models with a different number of cycles.

4.5.6 Concurrency

Modelers have to keep track of concurrent paths that need to be synchronized. AND-splits and OR-splits introduce new threads of control, so that the number of control tokens potentially increases by the number of the output degree minus one. The Token Split metric counts these newly introduced tokens. Concurrent tokens from the initial marking are not considered.

Symbol	TS
Definition	The token split sums up the output-degree of AND-joins and OR-joins minus one.
Metric	$TS(G) = \sum_{c \in C_{or} \cup C_{and}} d_{out}(n) - 1$
Rationale	A process model with a high token split value should be more likely to contain errors since it introduces a high degree of parallelism. A model with $TS = 0$ does not introduce new threads of execution after instantiation. ⁴
Limitations	There are models with the same token split value but which differ in comprehensibility, e.g. if one is structured.
Hypothesis	An increase in $TS(G)$ should imply an increase in error probability of the overall model.
Related Work	The maximum number of tokens was proposed by <i>Lee and Yoon</i> in [244]. While that approach is appealing, it is more computation intense than the token split metric. In <i>Nissen</i> [320], the concept of parallelism is captured by an approximation as number of nodes divided by diameter assuming that all splits introduce parallelism. <i>Morasca</i> proposes concurrent contribution which is related to token splits [308]. <i>Balasubramanian and Gupta</i> relate parallel nodes to nodes in total [37].

⁴ There may still be concurrency due to multiple start events.

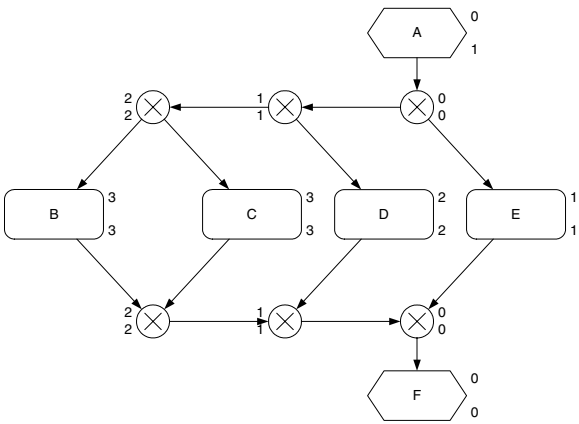


Figure 4.6. EPC with depth values next to nodes (top: in-depth, bottom: out-depth)

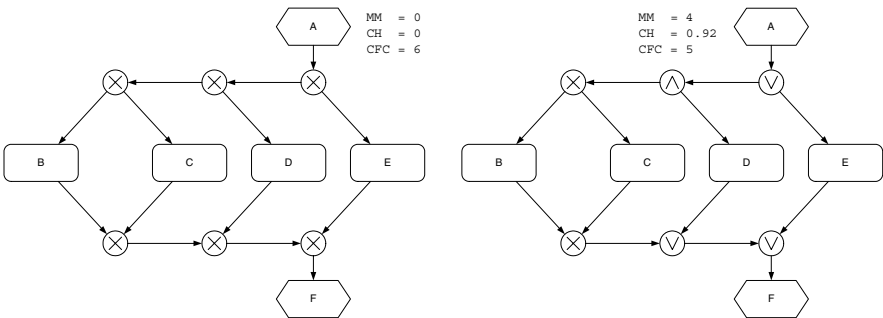


Figure 4.7. Two EPCs with different connector types

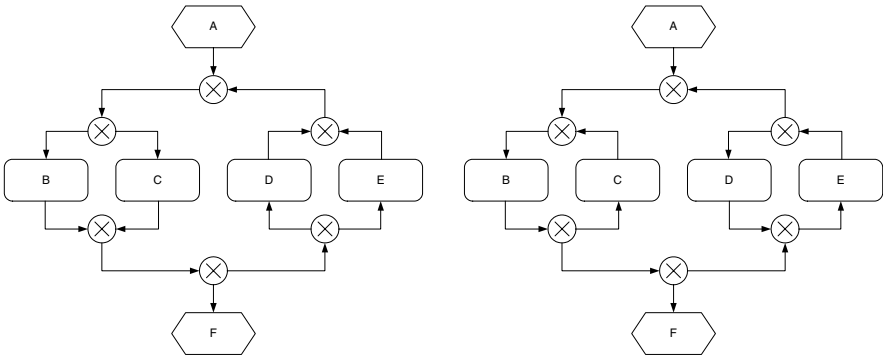


Figure 4.8. Two EPCs, one with nested cycles

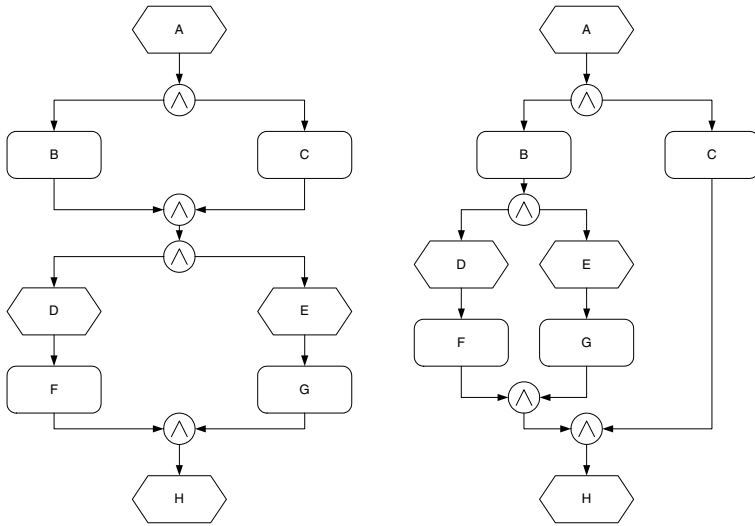


Figure 4.9. Two EPCs with the same token split values

Figure 4.9 illustrates that the token split gives an upper bound for the number of tokens in a model assuming boundedness. The EPC on the left-hand side contains two blocks of concurrency in sequence. Its maximum number of tokens is, therefore, lower than the token split value. In the model on the right-hand side the two blocks are nested. In this case, the maximum number of tokens matches the token split value.

The set of presented metrics reveals that there are several independent factors that presumably affect error probability. Approaches trying to squeeze the complexity of a process model⁵ into a single metric consequently seem doomed to fail. A similar observation is made by *Zuse* [474] who describes software complexity as a multi-dimensional phenomenon. In the subsequent section, we revisit the example EPC from Figure 2.1 to illustrate the metrics.

4.6 Calculating Metrics

In Section 2.1, we introduced the example of a loan request EPC taken from [325]. Table 4.1 summarizes the different metrics and Figure 4.10 illustrates which nodes and arcs contribute to the more elaborate metrics. Since the different count metrics, for size in particular, can be easily read from the model, we focus on those that need to be calculated from the process graph (separability, sequentiality, structuredness, depth, cyclicity and diameter).

The *separability ratio* Π depends on the identification of cut vertices (articulation points); those nodes whose deletion breaks up the graph in two or more disconnected

⁵ The complexity of a process model can be defined as the degree to which a process model is difficult to analyze, understand or explain (see [194]).

Table 4.1. Metrics derived from the EPC example

Size S_N	27	density Δ	0.040
Size S_E	11	density CNC	1.074
Size S_{ES}	1	av. connector degree $\overline{d_C}$	3
Size $S_{E_{Int}}$	8	max. connector degree $\widehat{d_C}$	3
Size S_{EE}	2	separability Π	0.440
Size S_F	8	sequentiality Ξ	0.345
Size S_C	8	structuredness Φ	0.556
Size $S_{S_{XOR}}$	3	depth Λ	1
Size $S_{J_{XOR}}$	2	mismatch MM	8
Size $S_{S_{AND}}$	0	heterogeneity CH	0.819
Size $S_{J_{AND}}$	2	control flow complexity CFC	8
Size $S_{S_{OR}}$	0	cyclicity CYC	0.259
Size $S_{J_{OR}}$	1	token splits TS	2
Size S_A	29		
Size $diam$	14		

components. Figure 4.10 displays articulation points with a letter A written next to the top left-hand side of the node. For example, if the function *record loan request* is deleted the start event is no longer connected with the rest of the process model. There are eleven articulation points in total yielding a separability ratio of $11/(27 - 2) = 0.440$. Note that start and end events do not belong to the set of articulation points since their removal does not increase the number of separate components.

The *sequentiality ratio* Ξ is calculated by relating the number of sequence arcs (arcs that connect functions and events) to the total number of arcs. Figure 4.10 highlights sequence arcs with an s label. There are ten sequence arcs and 29 arcs altogether which results in a sequentiality ratio of $10/29 = 0.345$. The degree of *structuredness* Φ relates the size of a reduced process model to the size of the original one. Figure 4.10 shows those elements with a cross on the left-hand side that are eliminated by reduction of trivial constructs. Other structured reduction rules are not applicable. Since 15 elements are deleted by reduction, the structuredness ratio is $1 - 15/27 = 0.556$. The *in-depth and out-depth* is also indicated for each node in Figure 4.10. The depth of a node is then the minimum of in-depth and out-depth. Several nodes have a depth of 1, which is a maximum, and therefore also the depth of the overall process. The *cyclicity* is based on the relation between number of nodes on a cycle and nodes in total. Figure 4.10 shows nodes on a cycle with a letter C written to the left-hand side bottom. There are seven such nodes yielding a cyclicity ratio of $7/27 = 0.259$. Figure 4.10 connects those 14 nodes that are on the *diameter* with a bold line.

We implemented the calculation of the various metrics as an extension to *xoEPC* (see Section 3.3.2). For each EPC that is analyzed by the program, the whole set of metrics is calculated and written to the entry for the model in the *errorreport.xml* file. We will use this feature in the following chapter for the analysis of an extensive collection of real-world EPC business process models.

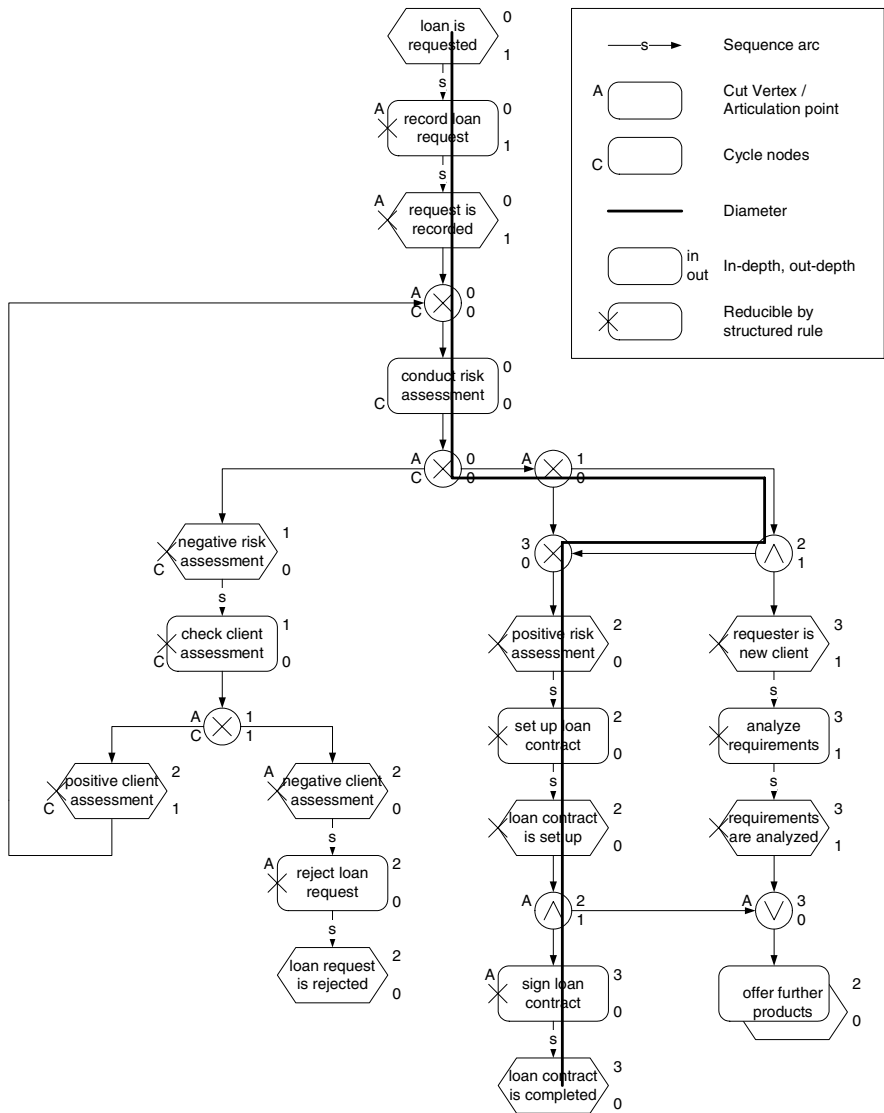


Figure 4.10. EPC example with sequence arcs, articulation points, cycle nodes, diameter, depth, and reducible nodes

4.7 Summary

In this chapter, we discussed the suitability of metrics for error prediction from a theoretical point of view. Revisiting related research in the area of network analysis, software measurement and metrics for business process models, we found that several aspects of process models were not yet combined in an overall measurement framework. Based on theoretical considerations, *we presented a set of 15 metrics related to size and 13 metrics that capture various aspects of the structure and the state space of the process model*. For each of the metrics we discussed a plausible connection with error probability and formulated respective hypotheses. In the following chapter, we will test these hypotheses for a large set of EPC business process models from practice using statistical methods.